

On-Policy Distillation 在视频扩散与 VLA 中的两种实现

AnyFlow 与 VLA-OPD 深度技术解读

OPD Papers 仓库·整理 by miracle-techlink

2026-05-15

Contents

1	摘要	3
2	1. 引言：为什么对比这两篇？	4
2.1	1.1 OPD 的统一范式	4
2.2	1.2 两个场景的本质差异	4
3	2. AnyFlow: 视频扩散中的 On-Policy Distillation	5
3.1	2.1 问题：Consistency 模型的 test-time scaling 困境	5
3.1.1	根本原因	5
3.2	2.2 解决方案：Flow Map 公式 + On-Policy 蒸馏	5
3.2.1	阶段 1: Forward Flow Map Training	5
3.2.2	阶段 2: On-Policy Flow Map Distillation	6
3.3	2.3 核心创新：Flow Map Backward Simulation	6
3.3.1	实际操作	6
3.3.2	DMD 损失	6
3.3.3	算法 2 伪代码	7
3.4	2.4 训练成本分析（论文 Tab. 5）	7
3.5	2.5 主要结果	7
3.5.1	Text-to-Video (VBench, Tab. 3)	7
3.5.2	Image-to-Video (VBench-I2V, Tab. 4)	8
3.6	2.6 AnyFlow 中 OPD 的关键设计要点	8
4	3. VLA-OPD: 机器人 VLA 模型的 On-Policy Distillation	9
4.1	3.1 问题：VLA 后训练的两难	9
4.1.1	SFT 的问题	9
4.1.2	RL (GRPO) 的问题	9
4.2	3.2 解决思路：用教师的 dense token-level 监督替代稀疏 reward	9
4.2.1	三阶段流水线（论文 Fig. 1 + Algorithm 1）	9
4.3	3.3 Token-level Reverse-KL 内在奖励	10
4.4	3.4 Group-Based Policy Gradient	10
4.5	3.5 为什么必须是 Reverse-KL?（消融关键）	10
4.6	3.6 三阶段算法伪代码（Algorithm 1）	11
4.7	3.7 主要结果	11
4.7.1	LIBERO 单臂操作（1-traj SFT init, 论文 Tab. 2）	11
4.7.2	LIBERO-Long 训练效率（论文 Fig. 2b）	12

4.7.3	RoboTwin2.0 双臂复杂操作 (论文 Tab. 3, 1000-traj SFT init)	12
4.7.4	抗遗忘 (Seen-Unseen Tradeoff, Fig. 3)	12
4.8	3.8 VLA-OPD 中 OPD 的关键设计要点	12
5	4. 横向对比：两种 OPD 实现的异同	13
5.1	4.1 共同点 (OPD 通用骨架)	13
5.2	4.2 不同点	13
5.2.1	4.2.1 “Trajectory” 的本质不同	13
5.2.2	4.2.2 Teacher 信号的传递方式	13
5.2.3	4.2.3 训练成本来源	13
5.2.4	4.2.4 解决遗忘的机制	13
5.2.5	4.2.5 与 RL 的关系	14
5.3	4.3 共同的失败前提 (两篇都隐含)	14
6	5. 启示与讨论	15
6.1	5.1 OPD 的可迁移设计模板	15
6.2	5.2 视频/扩散 → 这条路是否能继续?	15
6.3	5.3 VLA → 长程任务与 sim2real	15
6.4	5.4 共同的研究空白	15
7	6. 关键公式速查表	16
7.1	AnyFlow	16
7.2	VLA-OPD	16
8	7. 参考资料	16
8.1	主论文	16
8.2	OPD 范式来源	16
8.3	视频扩散蒸馏	17
8.4	VLA 后训练	17
8.5	仓库内相关解读 (references/)	17

1 摘要

On-Policy Distillation (OPD) 自 2025 年 10 月由 Thinking Machines Lab 系统化提出后，迅速从 LLM 后训练扩展到 非自回归生成模型 和 具身智能 (Embodied AI) 等场景。本文聚焦两篇代表性工作：

论文	任务	模型类型	蒸馏方式
AnyFlow (NVIDIA, 2026)	任意步视频生成	非自回归 Flow Map 扩散	DMD-style reverse-KL + Flow Map Backward Simulation
VLA-OPD (HKUST-GZ, 2026)	机器人操作	Vision-Language-Action	Token-level Reverse-KL + Group Policy Gradient

两者乍看属于完全不同的领域：一个是连续时间的视频扩散过程，一个是离散动作 token 的机器人策略。但深入剖析后会发现，它们共享同一套 **on-policy 采样 + dense 教师监督 + reverse-KL 优化** 的核心配方，区别仅在于“轨迹”和“reward 计算”在各自模态下的具体实例化。

本文系统地拆解二者的实现差异，并提炼出 OPD 在不同模态下的可迁移设计原则。

2 1. 引言：为什么对比这两篇？

2.1 1.1 OPD 的统一范式

经典 OPD 三要素（Thinking Machines Lab, Oct 2025）：

1. 学生 **on-policy** 采样 —— student 在自身策略下生成轨迹 $\tau \sim \pi_\theta$ 。
2. 教师密集打分 —— 强 teacher π_{teacher} 在 student 自己经过的状态上提供 token 级监督。
3. **Reverse-KL** 优化 —— 最小化 $D_{KL}(\pi_\theta \parallel \pi_{\text{teacher}})$ ，具备 mode-seeking 性质，“unhackable”。

LLM 场景下这三步非常直观：rollout 一段文本 → 教师算每个 token 的 logprob → 反向 KL 作为 reward。

但在 视频扩散 和 机器人 VLA 上，这三步分别变成了什么？

2.2 1.2 两个场景的本质差异

维度	AnyFlow (视频扩散)	VLA-OPD (机器人)
输出空间	连续 latent $\mathbf{z}_t \in \mathbb{R}^d$	离散动作 token $a_t \in \mathcal{A}$
时间结构	扩散步骤 $t \in [0, T]$	控制 horizon $t = 0, \dots, T$
Trajectory	ODE 采样路径 $\mathbf{z}_T \rightarrow \mathbf{z}_0$	观测-动作序列 $(o_0, a_0, \dots, o_T, a_T)$
Reward 来源	DMD 风格 score difference $s_{\text{real}} - s_{\text{fake}}$	教师 logits 与 student logits 的 logratio
Teacher	bidirectional 视频扩散基座 (Wan2.1)	RL 训出的 VLA expert (SimpleVLA-RL)
Backward signal	通过 Flow Map shortcut chain 反传	Token-level group policy gradient

下面分别详细展开，再做横向归纳。

3 2. AnyFlow: 视频扩散中的 On-Policy Distillation

【论文】**AnyFlow: Any-Step Video Diffusion Model with On-Policy Flow Map Distillation** Yuchao Gu, Guian Fang, Yuxin Jiang, Weijia Mao, Song Han, Han Cai, Mike Zheng Shou NVIDIA · Show Lab @ NUS · MIT · arXiv 2605.13724 (May 2026) 项目主页: <https://nvlabs.github.io/AnyFlow>

3.1 2.1 问题: Consistency 模型的 test-time scaling 困境

视频扩散模型推理代价巨大 (Wan2.1-T2V-14B 在 50×2 NFES 下生成 5 秒 480×832 视频要数十秒到几分钟)。少步蒸馏方法的两个主流路线:

- **Consistency Models (CM, rCM, sCM)**: 学一个映射 $\mathbf{z}_t \rightarrow \mathbf{z}_0$, 少步生成质量好。
- **Self-Forcing**: 用于因果视频扩散。

但 AnyFlow 论文 (Fig. 1) 指出一个反直觉现象: **Consistency-distilled** 模型在采样步数变多时质量反而下降。例如 rCM-14B 在 4 NFES = 83.73 (VBench), 到 32 NFES 反而降到 ~ 75 。

3.1.1 根本原因

Consistency 蒸馏把原本的 PF-ODE 轨迹替换成了 consistency 采样轨迹, 破坏了 ODE 的 **test-time scaling** 性质。多步采样时需要反复 endpoint projection + re-noise 来构造中间状态, 这些 re-noised 状态偏离了 base PF-ODE, 造成 trajectory drift (Fig. 2(a))。

3.2 2.2 解决方案: Flow Map 公式 + On-Policy 蒸馏

AnyFlow 的两步走:

3.2.1 阶段 1: Forward Flow Map Training

不再学 $\mathbf{z}_t \rightarrow \mathbf{z}_0$ 的端点映射, 而是学 **任意时间对** 之间的转移算子:

$$\mathbf{f}_\theta(\mathbf{z}_t, t, r) \approx \mathbf{z}_r, \quad 1 \geq t \geq r \geq 0$$

满足边界条件 $\mathbf{f}_\theta(\mathbf{z}_t, t, t) = \mathbf{z}_t$ 和组合性 $\Phi_{q \leftarrow r} \circ \Phi_{r \leftarrow t} = \Phi_{q \leftarrow t}$

基础目标 (MeanFlow):

$$\mathcal{L}(\theta) = \mathbb{E} \left[\left\| \mathbf{u}_\theta(\mathbf{z}_t, r, t) - \text{sg}(\mathbf{u}_{\text{tgt}}) \right\|_2^2 \right]$$

$$\mathbf{u}_{\text{tgt}} = \mathbf{v}(\mathbf{z}_t, t) - (t - r) \frac{d\mathbf{u}_\theta(\mathbf{z}_t, r, t)}{dt}$$

由于 JVP 在 FSDP 下不友好, 用有限差分近似 (只要 2 次 forward) :

$$\frac{d}{dt} \mathbf{u}(\mathbf{z}_t, r, t) \approx \frac{\mathbf{u}(\mathbf{z}_{t+\Delta t}, r, t + \Delta t) - \mathbf{u}(\mathbf{z}_{t-\Delta t}, r, t - \Delta t)}{2\Delta t}$$

工程细节（关键 hyperparameter，论文消融）：

- **Interpolated Timestep Conditioning**: $g \cdot \text{emb}(t) + (1 - g) \cdot \text{emb}'(r)$, $g = 0.25$ (不要用 zero-init, 否则 norm 飙升导致过饱和, 见 Fig. 11)
- **Time Sampler**: 先采 t, r , 再 $t = \max, r = \min$; 权重 $w(t) = \text{Beta}(2, 1.5)$ 性能最佳 (Tab. 10)
- **Guidance-Fused Training**: 把 CFG 直接融进预测 $\mathbf{u} = \frac{1}{g}(\mathbf{u}_c - (1 - g) \text{sg}(\mathbf{u}_\emptyset))$
- **Adaptive Loss Reweighting**: 自适应权重 $w_{t,r}$, 边界 $t = r$ 取 50% 训练 batch 做动态归一化

3.2.2 阶段 2: On-Policy Flow Map Distillation

这一步才是 **OPD** 在视频扩散中的体现。

总体目标: student 自己 rollout, 教师在 student 走过的轨迹上给 DMD-style reward。

3.3 2.3 核心创新: Flow Map Backward Simulation

为什么 Consistency Backward Simulation (Self-Forcing / rCM 使用) 不够好? 因为:

1. **重新加噪 (re-noising)**: consistency 模型只能到 $\mathbf{z}_0$, 需要 re-noise 回中间状态 $\rightarrow$ 偏离 PF-ODE
2. **训练成本与采样步数线性**: 模拟 N 步轨迹需要 N 次 forward 全部带梯度

AnyFlow 的洞察: **Flow Map** 模型本身就具备组合性:

$$\mathbf{f}_\theta(\mathbf{z}_t, t, q) \approx \mathbf{f}_\theta(\mathbf{f}_\theta(\mathbf{z}_t, t, r), r, q), \quad t > r > q$$

所以可以把整条 Euler rollout 分解为 **少数 shortcut 段**, 每段单独算梯度, 剩下的用 detach。

3.3.1 实际操作

对于目标采样预算 N 步:

1. 沿采样轨迹 **采样一个中间 timestep** t , 设下个 timestep $r = t - T/N$
2. 把 $T \rightarrow 0$ 轨迹分为 **三段**: $T \rightarrow t$ (shortcut), $t \rightarrow r$ (target transition), $r \rightarrow 0$ (shortcut)
3. 三段都通过 flow map 算 (都带 gradient):
 - $\mathbf{z}_t = \mathbf{f}_\theta(\mathbf{z}_T, c, T, t) \leftarrow$ shortcut, with grad
 - $\mathbf{z}_r = \mathbf{f}_\theta(\mathbf{z}_t, c, t, r) \leftarrow$ target step, with grad
 - $\mathbf{z}_0 = \mathbf{f}_\theta(\mathbf{z}_r, c, r, 0) \leftarrow$ shortcut, with grad
4. 在 $\mathbf{z}_0$ 上计算 KL gradient, 反传到整条链

3.3.2 DMD 损失

得到 student 自生成的 $\hat{\mathbf{z}}_0$ 后, 在 $t \in [0, T]$ re-noise 得 $\mathbf{z}_t = (1 - t)\hat{\mathbf{z}}_0 + t\epsilon$, DMD 梯度:

$$\nabla_\theta \mathcal{L}_{\text{DMD}} = -\mathbb{E}_{t, \mathbf{z}} \left[(s_{\text{real}}(\mathbf{z}_t, t) - s_{\text{fake}}(\mathbf{z}_t, t)) \frac{\partial \mathbf{f}_\theta(\mathbf{z})}{\partial \theta} \right]$$

s_{real} 来自 teacher (Wan2.1-T2V), s_{fake} 是一个学习的辅助模型 (DMD 标准做法)。

【提示】**OPD** 的视角解读: $s_{\text{real}} - s_{\text{fake}}$ 等价于 reverse-KL 散度的 score-based 近似。student 自己生成 $\hat{\mathbf{z}}_0 \rightarrow$ 教师在这个 student 自身经过的 $\mathbf{z}_t$ 状态上给打分 (“你这一步应该走到哪”) $\rightarrow$ 反传到所有 shortcut。这就是 **on-policy + dense reward** 的 reverse-KL 优化。

3.3.3 算法 2 伪代码

```
while training:
    # Sample inference step
    s ~ Uniform(1, 2, ..., T)
    # Sample gradient step
    t ~ Uniform(1, 2, ..., s)
    r = t - T/s
    # Sample initial noise
    z_T = randn_like(x)

    # Flow Map Backward Simulation
    z_t = fn(z_T, c, T, t) # With Grad
    z_r = fn(z_t, c, t, r) # With Grad
    z_0 = fn(z_r, c, r, 0) # With Grad

    # DMD-style on-policy update
    update theta via distribution matching loss
    using z_0 and re-noise at [0, T]
```

3.4 2.4 训练成本分析 (论文 Tab. 5)

	4 步 OPD	16 步 OPD
Consistency Backward Simulation	45.9 s/iter	93.8 s/iter
Flow Map Backward Simulation	53.1 s/iter (+15.7%)	53.1 s/iter (-43.4%)

关键观察: 步数越多, **Flow Map** 越省。因为 shortcut 把 N 步的反传压缩成了 3 段。在 16 步上比 consistency 路线快 43%-47%。

3.5 2.5 主要结果

3.5.1 Text-to-Video (VBench, Tab. 3)

模型	NFEs	Quality	Semantic	Total
Wan2.1-T2V-14B (Teacher)	50×2	85.77	75.58	83.74
rCM-Wan2.1-T2V-14B	4	85.47	76.72	83.73
AnyFlow-Wan2.1-T2V-14B	4	85.70	77.38	84.04
AnyFlow-Wan2.1-T2V-14B	32	85.76	77.44	84.10

模型	NFEs	Quality	Semantic	Total
AnyFlow-FAR-Wan2.1-14B (Causal)	4	85.82	76.97	84.05
AnyFlow-FAR-Wan2.1-14B (Causal)	32	86.12	77.55	84.41

关键观察：**AnyFlow** 在 **4** 步达到 **SOTA** 同时，**32** 步还在继续涨，rCM 32 步则倒退。这就是“any-step”的核心价值。

3.5.2 Image-to-Video (VBench-I2V, Tab. 4)

AnyFlow-FAR-Wan2.1-14B 用 **4 NFEs** 拿到 **87.87**，与原 Wan2.1-I2V-14B 用 50×2 NFEs 的 87.71 持平。

3.6 2.6 AnyFlow 中 OPD 的关键设计要点

1. 学生的 **trajectory** 不是离散动作序列，而是连续 **latent** 上的 **ODE** 路径
2. 教师 **supervision** 通过 **DMD** 的 **score difference** 实现（即 reverse-KL 的 score-based 估计）
3. **Backward** 通过 **flow map** 的 **shortcut** 组合性高效进行——不需要展开整条 N 步链
4. 保留 **PF-ODE** 性质，因此 OPD 后仍可任意步采样（distinguishing feature vs consistency）

4 3. VLA-OPD: 机器人 VLA 模型的 On-Policy Distillation

【论文】VLA-OPD: Bridging Offline SFT and Online RL for Vision-Language-Action Models via On-Policy Distillation Zhide Zhong, Haodong Yan, Junfeng Li, Junjie He, Tianran Zhang, Haoang Li HKUST (GZ) ·arXiv 2603.26666 (Mar 2026) 项目主页: <https://irpn-lab.github.io/VLA-OPD/>

4.1 3.1 问题: VLA 后训练的两难

预训练 VLA 模型 (OpenVLA, π_0 , NORA, RDT 等) 在通用泛化上不错, 但部署到具体任务往往不够稳。两条主流后训练路线:

范式	采样	信号	Few-Demo	收敛	抗遗忘	鲁棒性
Offline SFT	Off-policy	Dense	✗	Fast	✗	✗
Online RL (GRPO)	On-policy	Sparse	✓	Slow	✓	✓
VLA-OPD	On-policy	Dense	✓	Fast	✓	✓

4.1.1 SFT 的问题

- **Distribution Shift:** 在专家状态 D_{expert} 上训练, 部署时遇到 OOD 必崩
- **Catastrophic Forgetting:** 激进的参数更新破坏预训练通用能力

4.1.2 RL (GRPO) 的问题

- **Feedback Sparsity:** 机器人任务通常只在 episode 结束时给一个 binary 信号 $R(\tau) \in \{0, 1\}$
- **High Variance:** 优化方差大, 需要大量交互数据

4.2 3.2 解决思路: 用教师的 dense token-level 监督替代稀疏 reward

VLA-OPD 把 VLA 后训练 reformulate 为 “在 **student** 自生成轨迹上做密集监督”。

4.2.1 三阶段流水线 (论文 Fig. 1 + Algorithm 1)

4.2.1.1 Phase 1: On-Policy Sampling (Exploration) 学生 π_θ 在环境中自己跑:

$$\mathcal{D}_k = \{\tau \mid \tau = (s_0, a_0, s_1, a_1, \dots, s_T)\}, \quad a_t \sim \pi_{\theta_k}(\cdot | s_t), s_{t+1} \sim \mathcal{P}(\cdot | s_t, a_t)$$

关键: **states** 来自 **student** 自己的诱导分布 $d^{\pi_{\theta_k}}$, 包含大量 “失败状态” s_{err} 。这恰好是 SFT 没办法覆盖的 OOD 部分。

4.2.1.2 Phase 2: Dense Teacher Labeling (Correction) 冻结的教师 π_{tea} 在 student 走过的每个 state 上提供动作分布:

$$q_t(a) = \pi_{\text{tea}}(a | s_t)$$

教师 只打分，不在环境中执行（这点至关重要 —— 保证 on-policy）。

4.2.1.3 Phase 3: Mode-Seeking Optimization (Update) 用 **token-level Reverse-KL** 作为内在 reward 更新学生。

4.3 3.3 Token-level Reverse-KL 内在奖励

目标：

$$\max_{\theta} \mathcal{J}(\theta) = \mathbb{E}_{s \sim \pi_{\theta}} [-D_{\text{KL}}(\pi_{\theta}(\cdot|s) \parallel \pi_{\text{tea}}(\cdot|s))]$$

token 级 reward（即“内在 reward”）：

$$r_t^{\text{OPD}}(s_t, a_t) = -(\log \pi_{\theta}(a_t|s_t) - \log \pi_{\text{tea}}(a_t|s_t)) = -\log \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\text{tea}}(a_t|s_t)}$$

直觉：当 student 动作分布与 teacher 一致时 reward 接近 0；偏差大时 reward 显著为负，作为惩罚。注意：在计算梯度更新时，对 **student** 的 **logprob** $\log \pi_{\theta}$ 应用 **stop_gradient**，避免梯度路径混乱。

4.4 3.4 Group-Based Policy Gradient

为了降低 on-policy 梯度方差，仿照 GRPO 做 group sampling：对每条指令 s 采 G 条轨迹 $\{\tau_1, \dots, \tau_G\}$ ，梯度：

$$\nabla_{\theta} \mathcal{J}(\theta) \approx \frac{1}{G} \sum_{i=1}^G \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_{t,i}|s_{t,i}) \cdot r_t^{\text{OPD}}(s_{t,i}, a_{t,i})$$

vs 标准 GRPO: GRPO 的 advantage 是 group-normalized outcome reward；VLA-OPD 直接用 raw Reverse-KL 作为 advantage signal，**收敛更直接地朝着 teacher 的 optimal mode 走。**

4.5 3.5 为什么必须是 Reverse-KL?（消融关键）

论文给出三种对齐目标的对比（Fig. 4, RoboTwin2.0 Beat Block Hammer）：

目标	性质	后果
Forward KL $D_{KL}(\pi_{\text{tea}} \parallel \pi_{\theta})$	Mode-covering	Entropy Explosion — student 被迫覆盖 teacher 在 OOD 上的全部不确定性
Hard CE (DAgger 风格 argmax)	离散 0/1	Premature Entropy Collapse — 丢掉 dark knowledge，多模决策点剧烈振荡

目标	性质	后果
Reverse KL ✓	Mode-seeking	Bounded Stability — 由于 zero-forcing 性质, 只要 student 落在 teacher 高概率支持集内就不罚, 自动过滤教师 OOD epistemic uncertainty

实测训练曲线 (论文 Fig. 4a) : - Forward KL: 严重的 “performance valley”, 前期 success rate 暴跌 50%+ - Hard CE: 完全收敛失败, 平台在最低成功率 - **Reverse KL**: 稳定上升

4.6 3.6 三阶段算法伪代码 (Algorithm 1)

Input: 学生 π_θ (从 1-traj SFT 初始化), 教师 π_{tea} , group size G

```

while not converged do
  Sample batch  $\{o_j\}$  from  $D_{\text{prompt}}$ 
  for each prompt  $o$  do
    // Phase 1: Group sampling (on-policy)
    Generate  $G$  trajectories  $\{\tau_1, \dots, \tau_G\}$  using  $\pi_\theta(\cdot|o)$ 
    for each trajectory  $\tau_i$  do
      // Phase 2: Dense teacher labeling
      for each timestep  $t$  in  $\tau_i$  do
        Query  $\pi_\theta(\cdot|s_{\{t,i\}})$ ,  $\pi_{\text{tea}}(\cdot|s_{\{t,i\}})$ 
         $r_t = -(\log \pi_\theta(a_{\{t,i\}}|s_{\{t,i\}}) - \log \pi_{\text{tea}}(a_{\{t,i\}}|s_{\{t,i\}}))$ 
      end
    end
    // Phase 3: Group-based policy gradient
     $\nabla J \approx (1/(B \cdot G)) \sum_j \sum_i \sum_t \nabla \log \pi_\theta(a_{\{t,i\}}|s_{\{t,i\}}) \cdot r_t$ 
     $\theta \leftarrow \theta + \alpha \cdot \nabla J$ 
  end
end
end

```

4.7 3.7 主要结果

4.7.1 LIBERO 单臂操作 (1-traj SFT init, 论文 Tab. 2)

方法	Spatial	Object	Goal	Long	Avg
Teacher: SimpleVLA-RL	94.2	96.1	94.6	90.7	93.9
OpenVLA-OFT (Student init)	63.6	54.9	59.6	17.3	48.9
VLA-OPD (Distill only)	84.3	93.8	92.5	78.9	87.4
VLA-OPD (Distill + GRPO)	93.4	95.3	94.5	90.2	93.4

只用 1 条专家轨迹 + 蒸馏, 平均成功率从 **48.9% → 87.4%**, 几乎追平了用 50 条 demo 训练的 OpenVLA。进一步加 GRPO 微调可以达到 **93.4%**, 与 teacher 的 93.9% 持平。

4.7.2 LIBERO-Long 训练效率 (论文 Fig. 2b)

VLA-OPD 在 **50** 步 内达到 ~80% 成功率, GRPO 基线需要 **150** 步 —— **3× speedup**。

4.7.3 RoboTwin2.0 双臂复杂操作 (论文 Tab. 3, 1000-traj SFT init)

方法	Pick Dual Bottles	Place Empty Cup	Handover Block	Stack Bowls Tw
Teacher: SimpleVLA-RL	68.3	94.2	57.8	75.8
π_0	50.0	60.0	39.0	53.0
RDT	18.0	42.0	26.0	42.0
OpenVLA-OFT (Student init)	29.7	77.3	33.1	40.6
VLA-OPD (Distill only)	66.4	90.6	52.3	75.0

双臂任务比单臂难得多。从 45.2% → 71.1% (几乎追平 teacher 74.0%), 且没有用任何 RL fine-tune。

4.7.4 抗遗忘 (Seen-Unseen Tradeoff, Fig. 3)

固定 seen task 微调, 测 unseen task 成功率:

- **Offline SFT**: 严重塌方, Object task unseen 接近 0
- **Online RL**: 较好保留 unseen 能力
- **VLA-OPD**: 与 RL 相当或更好 (Object Task 1 上明显优于 RL)

4.8 3.8 VLA-OPD 中 OPD 的关键设计要点

1. 轨迹 = 环境交互的真实 (**o**, **a**) 序列 (不是文本, 不是 latent)
2. 教师只 **label** 不执行——避免破坏 student 的 on-policy 性质
3. **Reverse-KL** 是绝对刚需——Forward-KL 和 Hard-CE 都会失败
4. **Group-based gradient** 借鉴 GRPO 降方差, 但用 raw KL reward 替代 outcome-normalized advantage
5. **Distill + GRPO** 二阶段联合进一步提升

5 4. 横向对比：两种 OPD 实现的异同

5.1 4.1 共同点（OPD 通用骨架）

共同骨架	AnyFlow	VLA-OPD
学生自采样 冻结教师	自己跑 ODE 轨迹 $\mathbf{z}_T \rightarrow \mathbf{z}_0$ Wan2.1-T2V (base diffusion)	在环境中 rollout (o,a) 序列 SimpleVLA-RL (RL expert)
Dense 监督 Reverse-KL 优化	DMD score difference 通过 DMD 间接实现	Token-level logit difference 直接用 token-level reverse-KL
解决 exposure bias	用 student rollout 避开 train-test mismatch	用 student rollout 暴露 OOD 状态做主动纠错
Cold start	Stage 1 forward flow map training (SFT 性质)	1-traj SFT 初始化

5.2 4.2 不同点

5.2.1 4.2.1 “Trajectory” 的本质不同

- **AnyFlow**: 连续 latent 空间的 ODE 路径，每一“步”是个连续向量。Backward 通过自动微分链回传。
- **VLA-OPD**: 离散动作 token 序列，每一“步”是 categorical sampling。Backward 通过 policy gradient (log-derivative trick) 回传。

5.2.2 4.2.2 Teacher 信号的传递方式

- **AnyFlow**: 不显式写出 $\log \pi$ 的 ratio。通过 DMD 的两个 score model ($s_{\text{real}}, s_{\text{fake}}$) 估计 reverse-KL 梯度。这是因为图像/视频生成的“动作”是高维连续，不可能枚举所有 token。
- **VLA-OPD**: 直接写出 $\log \pi_{\theta} - \log \pi_{\text{tea}}$ 。VLA 模型把动作 tokenize 成有限词表，可以直接算 logprob。

5.2.3 4.2.3 训练成本来源

- **AnyFlow**: Cost 主要来自 backward through ODE chain。其 *Flow Map Backward Simulation* 的妙处在于把 N 步变 3 段。
- **VLA-OPD**: Cost 来自环境 rollout (机器人仿真) + teacher inference。其 *group sampling* 是分摊 prompt 上的方差。

5.2.4 4.2.4 解决遗忘的机制

- **AnyFlow**: 通过保留 base 的 flow field 实现下游 dataset 的 continued fine-tuning (Fig. 7-8)，不靠 OPD 本身防遗忘。
- **VLA-OPD**: OPD 本身就是抗遗忘的核心机制——把更新限制在 student 的 active policy manifold 内 (gentle alignment)。

5.2.5 4.2.5 与 RL 的关系

- **AnyFlow**: 与 RL 平行; OPD 是 RL 的高效替代。
- **VLA-OPD**: 与 RL 互补; “Distill + GRPO” 联合训练拿到最好结果。OPD 当 warmup, RL 做最后的优化。

5.3 4.3 共同的失败前提（两篇都隐含）

如果以下条件不满足，两种 OPD 都会失败：

1. **支持集重叠**: student 必须能采样到 teacher 高概率区域。AnyFlow 用 Stage 1 forward flow map training 保证, VLA-OPD 用 1-traj SFT 保证。
2. **Teacher 的“增量知识”真实存在**: 否则只是用大模型做 supervised distillation 的换皮。AnyFlow 的 teacher 是 50-step PF-ODE 完整结果, VLA-OPD 的 teacher 是 RL 训练过的 expert——都带“新知识”。
3. **避免 mode covering**: 两者都选择 reverse-KL, 避免 forward-KL 的熵爆炸。

6 5. 启示与讨论

6.1 5.1 OPD 的可迁移设计模板

无论模态，OPD 都可以套用以下“配方”：

1. 准备 student (base 模型) + teacher (更强的同族模型或专家)
2. Cold start: student 经过 mid-train / SFT 拉近与 teacher 的支持集
3. On-policy 采样: 让 student 走自己的"轨迹" (生成 / 控制 / 推理)
4. Teacher 在 student 轨迹上提供 dense 监督
 - 离散动作: token logits 直接比较
 - 连续生成: score function 或 distribution matching
5. Reverse-KL 优化, mode-seeking
6. Group sampling / shortcut decomposition 降低方差与成本

6.2 5.2 视频/扩散 → 这条路是否能继续?

AnyFlow 已经展示 OPD 可以替代 consistency 蒸馏的位置。后续可能的发展：

- **Long video 上 OPD**: 当前 AnyFlow 是 ~81 帧。长视频 + autoregressive 生成结合后，OPD 如何处理 exposure bias 累积？
- **3D / Multi-modal generation**: NeRF / 3D Gaussian Splatting 也可以套相同范式
- **Audio / TTS**: 流匹配 TTS (如 F5-TTS) 已有，OPD 应该直接可用

6.3 5.3 VLA → 长程任务与 sim2real

VLA-OPD 当前在 LIBERO / RoboTwin2.0 这类 simulator 上验证。开放问题：

- **真实机器人**: teacher labeling 是否可以从 RL-trained simulation expert 迁移到真实环境？
- **多阶段任务**: 50+ 步的复杂任务 (叠衣服等), dense reward 是否仍可靠? 回顾 Rethinking OPD 论文: “reward quality degrades with trajectory depth”, 长程任务可能要重新设计 teacher。
- **多 teacher**: DeepSeek-V4 已展示多专家 OPD, 机器人也可以 “多 teacher 路由”——pick-place 用一个 teacher, stack 用另一个。

6.4 5.4 共同的研究空白

1. **跨 tokenizer / 跨架构的 OPD**: HuggingFace 已有 General On-Policy Logit Distillation 但视频/VLA 上还没有
2. **OPD 的稳定性理论**: reverse-KL 在什么条件下保证收敛? (参考 Rethinking OPD 的失败模式分析)
3. **OPD 与 RLHF 的统一**: VLA-OPD 已展示 “Distill + GRPO” 联合训练效果好, 但还缺少理论指导

7 6. 关键公式速查表

7.1 AnyFlow

MeanFlow 目标:

$$\mathcal{L}(\theta) = \mathbb{E} [\|\mathbf{u}_\theta(\mathbf{z}_t, r, t) - \text{sg}(\mathbf{u}_{\text{tgt}})\|_2^2], \quad \mathbf{u}_{\text{tgt}} = \mathbf{v}(\mathbf{z}_t, t) - (t - r) \frac{d\mathbf{u}_\theta}{dt}$$

Interpolated timestep: $g \cdot \text{emb}(t) + (1 - g) \cdot \text{emb}'(r)$, $g = 0.25$

Flow Map 组合性: $\mathbf{f}_\theta(\mathbf{z}_t, t, q) \approx \mathbf{f}_\theta(\mathbf{f}_\theta(\mathbf{z}_t, t, r), r, q)$

DMD 梯度:

$$\nabla_\theta \mathcal{L}_{\text{DMD}} = -\mathbb{E}_{t, \mathbf{z}} \left[(s_{\text{real}}(\mathbf{z}_t, t) - s_{\text{fake}}(\mathbf{z}_t, t)) \frac{\partial \mathbf{f}_\theta}{\partial \theta} \right]$$

7.2 VLA-OPD

Reverse-KL 目标:

$$\max_{\theta} \mathcal{J}(\theta) = \mathbb{E}_{s \sim \pi_\theta} [-D_{\text{KL}}(\pi_\theta(\cdot|s) \parallel \pi_{\text{tea}}(\cdot|s))]$$

Token-level intrinsic reward:

$$r_t^{\text{OPD}}(s_t, a_t) = -\log \frac{\pi_\theta(a_t|s_t)}{\pi_{\text{tea}}(a_t|s_t)}$$

Group policy gradient:

$$\nabla_\theta \mathcal{J}(\theta) \approx \frac{1}{G} \sum_{i=1}^G \sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_{t,i}|s_{t,i}) \cdot r_t^{\text{OPD}}(s_{t,i}, a_{t,i})$$

8 7. 参考资料

8.1 主论文

1. **AnyFlow** — Gu et al., NVIDIA, arXiv 2605.13724 (2026)
 - 项目主页: <https://nvlabs.github.io/AnyFlow>
 - 代码: <https://github.com/NVlabs/AnyFlow>
2. **VLA-OPD** — Zhong et al., HKUST(GZ), arXiv 2603.26666 (2026)
 - 项目主页: <https://irpn-lab.github.io/VLA-OPD/>

8.2 OPD 范式来源

3. **On-Policy Distillation** — Kevin Lu, Thinking Machines Lab (Oct 2025) <https://thinkinngmachines.ai/blog/on-policy-distillation/>
4. **Rethinking OPD: Phenomenology, Mechanism, and Recipe** — THUNLP, arXiv 2604.13016 分析了 OPD 何时失败、为什么失败

8.3 视频扩散蒸馏

5. **rCM (sCM)** — Zheng et al., arXiv 2510.08431 (Score-regularized Continuous-time Consistency)
6. **Self-Forcing** — Huang et al., NeurIPS 2025 (causal video diffusion OPD)
7. **MeanFlow** — Geng et al., arXiv 2505.13447 (one-step generative modeling)
8. **DMD (Distribution Matching Distillation)** — Yin et al., NeurIPS 2024 / CVPR 2024
9. **Align Your Flow** — Sabour et al., NVIDIA, arXiv 2506.14603 (continuous-time flow map)

8.4 VLA 后训练

10. **OpenVLA-OFT** — Kim et al., arXiv 2502.19645
11. π_0 / $\pi_{0.5}$ / $\pi_{0.6}$ — Physical Intelligence
12. **LIBERO** — Liu et al., NeurIPS 2023
13. **RoboTwin2.0** — Chen et al., arXiv 2506.18088
14. **SimpleVLA-RL** — Li et al., arXiv 2509.09674 (teacher used in VLA-OPD)
15. **GRPO** — DeepSeek-R1, arXiv 2501.12948

8.5 仓库内相关解读 (references/)

- 05_thinking-machines_on-policy-distillation.md — 原始 OPD 博客
- 02_qingkeai_rethinking-opd.md — 失败模式深度分析
- 06_qingkeai_seven-papers-opd-deep-dive.md — 7 篇 OPD 论文串讲
- 07_qingkeai_three-schools-9-papers.md — 9 篇近期工作 (含 Video-OPD)

文档版本: v1.0 最后更新: 2026-05-15 所属仓库: [miracle-techlink/opd-papers](https://github.com/miracle-techlink/opd-papers)